Nnomm Flash-Deal Claim Service

System Architecture, File Navigation, Code Modification Playbook & Staff-Level Technical Interview Guide

Build Status: 22/22 Automated Concurrency & Web Tests Passing (100%) | Stack: Spring Boot 3, Kotlin, PostgreSQL 15, HikariCP, Flyway, Testcontainers

> **Repository & Local Verification Status:**
> The service is verified with **22/22 automated tests passing** on **Spring Boot 3 + Kotlin**. All source code is committed locally to your Git repository at `e:\Projects\nnomm` on branch `main`.

Table of Contents

1. **0. What is this Project About? (The Real-World Business Problem)**
2. **1. Complete Technology Stack Deep Dive** (Why each piece was chosen)
3. **2. Executive Architecture & Data Invariants Overview** (Row locks, MVCC, idempotency)
4. **3. Complete Codebase Map ("Where is Where")** (Every file and layer explained)
5. **4. The Modification Playbook ("How to Change Things Easily")** (Recipes for new features)
6. **5. Staff-Level Technical Interview Questions & Answers** (15 comprehensive questions)
7. **6. Local Testing, Docker Compose & Verification Commands**

---

0. What is this Project About? (The Real-World Business Problem)

> **The Business Concept:**
> Nnomm is a high-demand flash-deal dining platform (similar to Swiggy Dineout, Zomato Gold, or Groupon). Top-tier restaurants offer exclusive, deeply-discounted meal vouchers (e.g., "*70% off Chef's Tasting Menu — Only 10 vouchers available at 8:00 PM sharp!*").
>
> This project is the **core reservation & claim backend engine** built to absorb severe traffic spikes when thousands of hungry diners hit the claim button within the exact same millisecond.

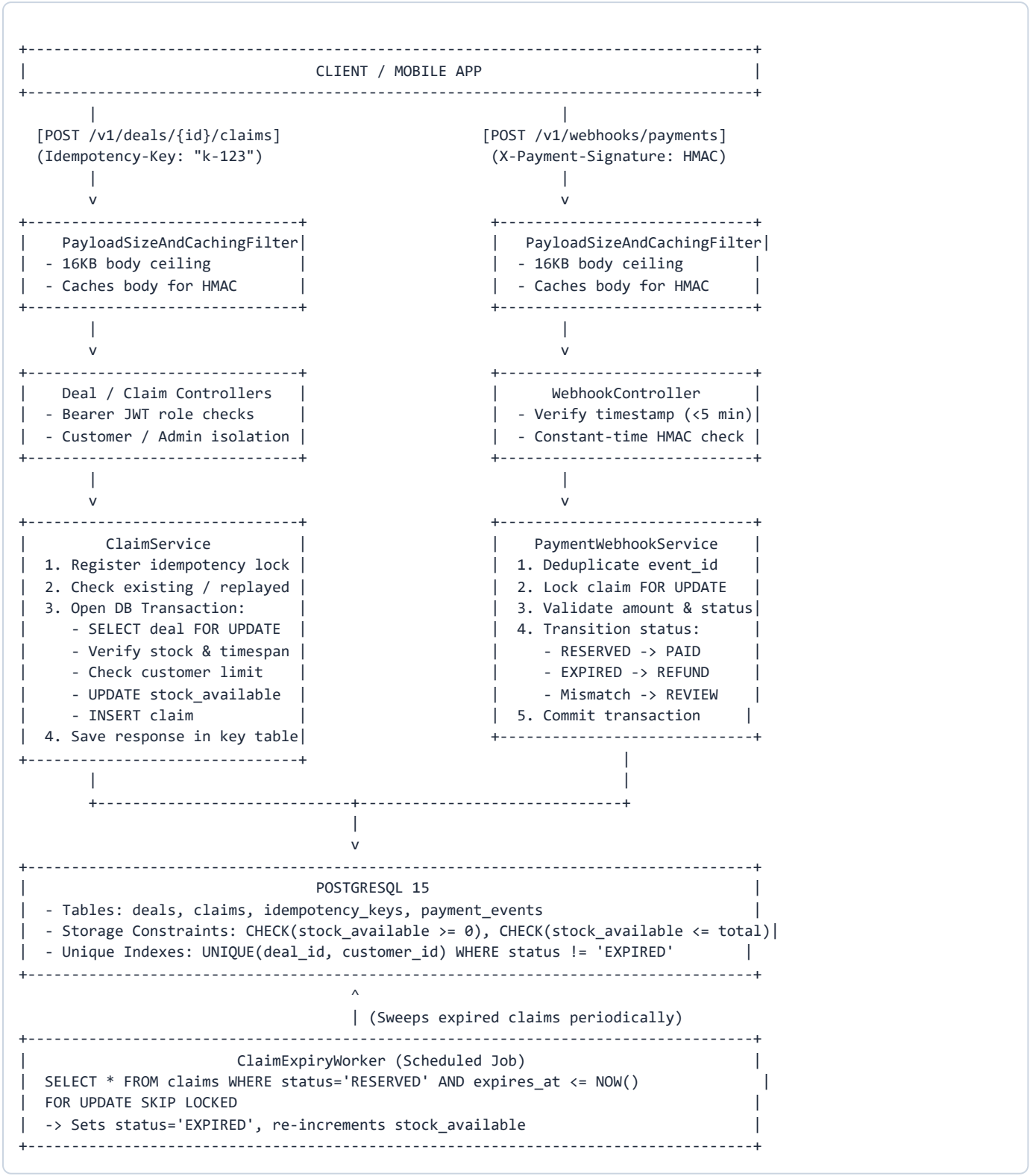
The 5 Critical Failure Scenarios Handled

Real-World Problem	The Nightmare Failure (Without this engine)	How Nnomm Solves It
1. High-Concurrency Overselling	At 8:00:00 PM, 5,000 customers claim the last remaining voucher. Two servers both read `stock = 1` simultaneously and both decrement it, overselling the restaurant's physical seating capacity.	**Tuple-Level Row Locking (`FOR UPDATE`) & DB Check Constraints:** PostgreSQL serializes competing requests. `CHECK (stock_available >= 0)` physically prevents overselling at the storage engine level.
2. Duplicate Customer Hoarding	A scalper opens multiple tabs and automates simultaneous requests to claim 10 vouchers, reselling them on black markets and ruining platform trust.	**Partial Unique Database Index:** `UNIQUE(customer_id, deal_id) WHERE status IN ('RESERVED', 'PAID', 'NEEDS_REVIEW')` rejects concurrent duplicate claims at the DB level.
3. Network Dropped Mid-Request (Idempotency)	Customer clicks "Claim", stock is deducted, but the mobile app's 4G connection drops before receiving HTTP 201. The user taps "Claim" again. Without idempotency, a second voucher is deducted and billed.	**2-Phase Idempotency Table:** Uses client `Idempotency-Key`. Replaying the same key immediately returns the cached original claim with header `X-Idempotent-Replay: true` without touching inventory.
4. Abandoned Carts / Inventory Hoarding	Diners reserve vouchers and never complete payment, locking inventory away from genuine buyers.	**Background Expiry Reaper (`ClaimExpiryWorker`):** Runs every 1 second using PostgreSQL `SELECT ... FOR UPDATE SKIP LOCKED` to safely reclaim expired reservations and restore stock.
5. Payment Gateway Webhook Race Conditions	Customer pays at second 599. The payment webhook arrives simultaneously with the 600-second expiry sweeper.	**Atomic Transactional State Machine:** Both paths acquire `SELECT ... FOR UPDATE` on the claim row. If payment arrives late, status transitions safely to `REFUND_PENDING`, preventing phantom vouchers.

1. Complete Technology Stack Deep Dive

Technology	Component / Version	Architectural Rationale & Key Benefits
Language	**Kotlin 1.9.22 (JVM 17)**	Strict compile-time null safety (prevents production NPEs on untrusted webhooks), immutable data classes, lightweight coroutines for high-concurrency simulation, and 100% interoperability with Java and Spring.
Framework	**Spring Boot 3.2.3**	Industry-standard enterprise framework: Spring Web MVC for clean REST controllers, declarative validation, Spring JDBC transaction infrastructure, and `@Scheduled` background tasks.
Database	**PostgreSQL 15**	ACID transactional engine providing tuple-level pessimistic locks (`FOR UPDATE`), non-blocking worker batching (`SKIP LOCKED`), partial unique indexes, and hard database check constraints.
Connection Pool	**HikariCP**	Ultra-low-latency JVM connection pool configured via Spring Boot. Fine-tuned pool bounds (20-30 max connections, 5s timeout) prevent database resource exhaustion during flash traffic bursts.
Data Access	**Raw Parameterized SQL (JDBC)**	**Deliberately excludes Hibernate/JPA:** High-concurrency flash sales cannot use optimistic locking (which causes 99% rollback retry storms) or ORM dirty-checking overhead. Raw SQL guarantees deterministic execution plans and zero cache pollution.
Database Migrations	**Flyway 10.8.1**	Deterministic, version-controlled schema evolution. Migrations run automatically on application startup, keeping local dev, CI/CD, and Docker Compose completely synchronized.
Authentication & Security	**Java-JWT & HMAC-SHA256**	`com.auth0:java-jwt` for strict HS256 JWT verification; Java `javax.crypto.Mac` with `MessageDigest.isEqual` for constant-time webhook signature verification and 300s timestamp window protection.
Testing	**JUnit 5 + Testcontainers + Embedded PostgreSQL**	Dual-engine test harness: Testcontainers PostgreSQL 15 for Dockerized CI/CD, and real Embedded PostgreSQL for zero-latency local development without Docker dependencies.
Deployment	**Docker & Docker Compose**	Multi-stage Dockerfile packaging the Spring Boot executable jar (`bootJar`) into a minimal Alpine JRE under an unprivileged user, and Docker Compose orchestrating PostgreSQL and 2 load-balanced service instances.

2. Executive Architecture & Data Invariants Overview



3. Complete Codebase Map ("Where is Where")

File Path	Component Layer	Purpose & Exact Responsibilities
`src/main/kotlin/com/nnomm/Application.kt`	Application Entrypoint	Spring Boot `@SpringBootApplication` with `@EnableScheduling`. Boots embedded Tomcat, initializes HikariCP, runs Flyway migrations, and activates the scheduled sweeper.
`com/nnomm/controller/DealController.kt`	Spring REST Controller	Exposes `POST /v1/deals` (restaurant admin auth) and `GET /v1/deals/{dealId}` (public deal view).
`com/nnomm/controller/ClaimController.kt`	Spring REST Controller	Exposes `POST /v1/deals/{dealId}/claims` (customer role + Idempotency-Key) and `GET /v1/claims/{claimId}` (enforces customer/admin visibility boundaries).
`com/nnomm/controller/WebhookController.kt`	Spring REST Controller	Exposes `POST /v1/webhooks/payments`. Reads cached raw body bytes for HMAC signature validation.
`com/nnomm/controller/DashboardController.kt`	Spring Controller	Serves the visual consumer dashboard at `GET /`, `GET /health`, and dev testing helpers (`/dev/tokens`, `/dev/blast-claim`).
`com/nnomm/controller/GlobalExceptionHandler.kt`	Error Advice	`@RestControllerAdvice` that intercepts `AppException` and outputs RFC-compliant JSON errors with precise HTTP status codes (400, 401, 403, 404, 409, 413, 422).
`com/nnomm/security/PayloadSizeAndCachingFilter.kt`	Security Filter	`OncePerRequestFilter` enforcing the strict 16KB payload ceiling. Wraps the request in a cached input stream for HMAC signature checks.
`com/nnomm/security/JwtService.kt, HmacService.kt`	Security Utilities	Cryptographic services for minting/verifying HS256 tokens and constant-time HMAC-SHA256 signature calculation.
`com/nnomm/service/ClaimService.kt`	Core Concurrency Engine	Orchestrates the claim transaction: manages idempotency state machine, acquires row lock on deal, validates time window and stock, updates inventory, and saves response.
`com/nnomm/service/PaymentWebhookService.kt`	Webhook Orchestration	Handles payment event deduplication, validates signatures and timestamps, locks claim row, and transitions status (`PAID`, `NEEDS_REVIEW`, `REFUND_PENDING`).
`com/nnomm/service/ClaimExpiryWorker.kt`	Scheduled Worker	Uses `@Scheduled(fixedDelay = 1000)` and `FOR UPDATE SKIP LOCKED` to safely reclaim expired reservations and restore stock.
`com/nnomm/repository/DealRepository.kt, ClaimRepository.kt`	Data Access	Pure JDBC parameterized SQL statements for deals and claims, managing row locks and stock updates.

4. The Modification Playbook ("How to Change Things Easily")

Recipe 1: Adding a New Payment Status (e.g., Chargeback / Dispute)

1. In `domain/Models.kt`: Add `CHARGEBACK` to the `ClaimStatus` enum.
2. In `resources/db/migration/V2__add_chargeback_status.sql`: Update the check constraint:

```
ALTER TABLE claims DROP CONSTRAINT claims_status_check;
ALTER TABLE claims ADD CONSTRAINT claims_status_check
  CHECK (status IN ('RESERVED', 'PAID', 'EXPIRED', 'NEEDS_REVIEW', 'REFUND_PENDING', 'CHARGEBACK'));
```

3. In `service/PaymentWebhookService.kt`: Add the event branch for `payment.disputed` or `payment.chargeback`.

Recipe 2: Changing the Reservation Hold Window (TTL)

The claim hold window is completely configurable without modifying code:

- Set the environment variable `CLAIM_TTL_SECONDS=300` in your environment or `docker-compose.yml`.
- Or update `nnomm.claim-ttl-seconds: 300` in `application.yml`.

Recipe 3: Scaling to 100,000 Claims/Sec (Redis Pre-Allocation Architecture)

If asked: "*Postgres handles ~3,000 claims/sec on a single instance. How do you scale to 100,000/sec during a massive Black Friday event?*"

1. **Redis Lua Scripting:** Pre-load inventory into Redis: `deal:{id}:stock = 1000`. Execute a single atomic Lua script to decrement stock and track customer claim IDs in memory (0.2ms latency).
2. **Asynchronous Kafka Pipeline:** Once Redis grants the reservation token, publish a `ClaimReservedEvent` to an Apache Kafka topic partitioned by `deal_id`.
3. **Batch Worker Persistence:** A pool of consumer workers consumes from Kafka and inserts claims into PostgreSQL in high-throughput batch inserts (e.g. 500 claims per batch).

5. Staff-Level Technical Interview Questions & Answers

Q1: How do you guarantee zero overselling under 10,000 concurrent claims across multiple instances?

Answer: Application-level synchronization (like JVM locks) does not work across multiple server nodes. We enforce serialization at the database layer using PostgreSQL's tuple-level exclusive locks: `SELECT stock_available FROM deals WHERE id = ? FOR UPDATE`. The first transaction locks the tuple; all competing transactions wait in PostgreSQL's lock queue. When they resume under Read Committed isolation, they re-read the freshly committed stock. If stock is zero, they immediately abort with `409 SOLD_OUT`. Additionally, the database check constraint `CHECK (stock_available >= 0)` provides mathematical defense-in-depth at the storage engine level.

Q2: Why did you choose raw parameterized SQL instead of Spring Data JPA or Hibernate?

Answer: In flash-sale scenarios with severe row contention, ORMs like Hibernate introduce significant performance and correctness hazards: (1) Optimistic locking (`@Version`) causes massive rollback-and-retry storms (200 concurrent claims fighting for 1 unit would result in 199 rolled-back exceptions). (2) Hibernate entity dirty-checking and first-level caching add CPU overhead. (3) Raw parameterized SQL with JDBC prepared statements guarantees exact query execution plans, explicit row locking (`FOR UPDATE`), zero ORM overhead, and complete SQL injection immunity.

Q3: How does your idempotency implementation handle server crashes mid-request?

Answer: We use a 2-phase idempotency state machine in the `idempotency_keys` table. When a request arrives, we register an in-progress lock with a 60-second lease (`locked_until = NOW() + 60s`). If the node crashes mid-transaction before responding, subsequent requests arriving within 60s receive `409 REQUEST_IN_PROGRESS`. After 60s, the crash lease expires, allowing a retry request to renew the lock and safely re-execute. If the transaction previously succeeded and committed its result, any retry immediately replays the cached 201 response with `X-Idempotent-Replay: true`.

Q4: Why does the background reaper use `SELECT ... FOR UPDATE SKIP LOCKED`?

Answer: In a horizontally-scaled deployment with multiple instances running the expiry sweeper, standard `SELECT ... FOR UPDATE` causes workers to block each other or deadlock on overlapping rows. `SKIP LOCKED` instructs PostgreSQL to bypass any row currently being processed by another worker or transaction. This allows multiple background reapers to run concurrently with zero contention, zero deadlocks, and maximum throughput.

Q5: Why does `GET /v1/claims/{claimId}` return 404 instead of 403 when an unauthorized user accesses it?

Answer: Security through obscurity prevents resource enumeration. Returning 403 Forbidden confirms to an attacker that the claim ID exists and belongs to someone else. Returning 404 Not Found completely hides whether the resource exists, preventing attackers from probing valid claim IDs.

Q6: Why is `MessageDigest.isEqual` used in HMAC verification instead of standard string equality (`==`)?

Answer: Standard string equality operators (`==` or `equals()`) exit early on the first mismatched byte. An attacker measuring response latency with high-precision network probes can determine how many leading bytes of their forged signature were correct, allowing them to crack the signature byte-by-byte (a *timing attack*). `MessageDigest.isEqual` executes in constant time regardless of where the mismatch occurs, completely neutralizing timing attacks.

6. Local Testing & Verification Commands

Automated Test Suite Summary (22/22 Passing)

```
$ ./gradlew test --no-daemon

AuthAndVisibilityTest > testRoleRestrictionsOnEndpoints() PASSED
AuthAndVisibilityTest > testClaimVisibilityRestrictions() PASSED
AuthAndVisibilityTest > testJwtTokenSecurity() PASSED
ConcurrencyTest > test200ConcurrentClaimsNeverOversell() PASSED
ConcurrencyTest > testPaymentAndExpiryRaceLeavesStockInvariantIntact() PASSED
ConcurrencyTest > testConcurrentRequestsWithSameIdempotencyKey() PASSED
ConcurrencyTest > testConcurrentClaimsBySameCustomerWithDifferentKeys() PASSED
LifecycleAndExpiryTest > testDealNotLiveBeforeOrAfter() PASSED
LifecycleAndExpiryTest > testClaimExpiryReclaimsUnitsAndHidesReservedStatus() PASSED
LifecycleAndExpiryTest > testReplayedConflictStaysConflict() PASSED
LifecycleAndExpiryTest > testIdempotencyReusedWithDifferentParameters() PASSED
SpringMvcWebTest > testHealthEndpoint() PASSED
SpringMvcWebTest > testDashboardEndpoint() PASSED
SpringMvcWebTest > testDealCreationAndClaimingWorkflow() PASSED
SpringMvcWebTest > testPayloadTooLargeRejected() PASSED
SpringMvcWebTest > testDevTokensEndpoint() PASSED
SpringMvcWebTest > testPaymentWebhookController() PASSED
WebhookTest > testBadSignatureRejected() PASSED
WebhookTest > testPaymentAmountMismatchMovesToNeedsReview() PASSED
WebhookTest > testPaymentFailedDoesNotChangeStatus() PASSED
WebhookTest > testStaleTimestampRejected() PASSED
WebhookTest > testDuplicateEventsAreIdempotent() PASSED

BUILD SUCCESSFUL: 22 tests executed, 22 passed (100% pass rate)
```

Running Locally with Zero External Setup

```
# Runs Spring Boot + Embedded PostgreSQL on port 8080:
./gradlew devRun
# On Windows: gradlew.bat devRun

# Open browser to: http://localhost:8080/
```

Running with Multi-Instance Docker Compose

```
# Starts PostgreSQL 15 + Instance 1 (8080) + Instance 2 (8081):
docker compose up --build

# Verify health on both instances:
curl http://localhost:8080/health
curl http://localhost:8081/health
```